



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

PARADIGMAS DE PROGRAMACIÓN

UNIDAD 2

Facultad de Ingeniería
Universidad Nacional de Cuyo

2026

- 1 Maybe
- 2 Currificación
- 3 Parcialización
- 4 Orden superior
- 5 Evaluación Lazy
- 6 Ejemplificación
- 7 Monadas

Maybe

El tipo Maybe: ausencia de valor como dato

Definición conceptual

```
data Maybe a = Nothing | Just a
```

- **Nothing**: no hay resultado.
- **Just a**: sí hay un valor de tipo **a**.
- Permite escribir funciones **totales**: el caso problemático se representa explícitamente.

Aporte al paradigma funcional

- Evita centinelas como **-1** o **null**.
- Hace visibles los casos de falla en el tipo.
- Prepara la intuición de **cómputo con contexto**.

El tipo Maybe: ausencia de valor como dato

```
safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x
usar :: [Int] -> Int
usar xs = case safeHead xs of
    Nothing -> 0
    Just n   -> n + 1
```

Maybe en acción: composición segura

```
safeDiv :: Double -> Double -> Maybe
```

```
  ↪ Double
```

```
safeDiv _ 0 = Nothing
```

```
safeDiv x y = Just (x / y)
```

```
safeRoot :: Double -> Maybe Double
```

```
safeRoot x
```

```
  | x < 0      = Nothing
```

```
  | otherwise = Just (sqrt x)
```

```
proceso :: Double -> Double -> Maybe Double
```

```
proceso a b =
```

```
  case safeDiv a b of
```

```
    Nothing -> Nothing
```

```
    Just q   -> safeRoot q
```

Lectura

- La función no “lanza” nada: **devuelve un contexto.**
- El consumidor debe decidir qué hacer con ese contexto.

Conexión con lo que sigue

Más adelante veremos que las mónadas permiten **encadenar** este patrón sin repetir el **case** manualmente.

Currificación

Currificación

¿Qué diferencias hay entre las siguientes funciones?

```
minimo :: (Int, Int) -> Int
minimo (x,y) = if x >= y then x else y
```

```
minimoC :: Int -> Int -> Int
minimoC x y = if x >= y then x else y
```

Currificación

La función `minimoC` está currificada:

```
minimoC :: Int -> Int -> Int
```

Esto se interpreta como:

```
minimoC :: Int -> (Int -> Int)
```

Es decir, recibe un `Int` y devuelve una función que espera otro `Int`.

Currificación

Idea

Una función de “dos argumentos” se interpreta como:

$$A \rightarrow B \rightarrow C \equiv A \rightarrow (B \rightarrow C)$$

Primero recibe un **A** y devuelve una función que espera un **B**.

```
suma :: Int -> Int -> Int
suma x y = x + y
```

```
-- lectura equivalente
suma' :: Int -> (Int -> Int)
suma' = \x -> (\y -> x + y)
```

Relación con cálculo lambda

$$\lambda x. \lambda y. x + y$$

La currificación expresa funciones multiargumento con **abstracciones unarias encadenadas**. Aporta composición y reutilización.

Currificación

- Ejemplo de Currificación:

```
suma' :: Int -> (Int -> Int)
suma' x y = x + y
```

```
*Main> :type suma' 2
suma' 2 :: Int -> Int
*Main> :type suma' 2 3
suma' 2 3 :: Int
```

Currificación con tres argumentos

- Ejemplo de Currificación con tres argumentos:

```
mult :: Int -> (Int -> (Int -> Int))  
mult x y z = x * y * z
```

```
*ghci> :type mult 2  
mult 2 :: Int -> (Int -> Int)  
*ghci> :type mult 2 3  
mult 2 3 :: Int -> Int  
*ghci> :type mult 2 3 7  
mult 2 3 7 :: Int
```

Aplicación parcial

- Las funciones currificadas pueden aplicarse parcialmente. Por ejemplo,

```
*ghci> (suma' 2) 3
```

- Pueden definirse funciones usando aplicaciones parciales. Por ejemplo,

```
suc :: Int -> Int  
suc = suma' 1
```

- suc x es el sucesor de x. Por ejemplo, suc 2 es 3.
- Las funciones suma' y mult son currificadas.

Convenios para reducir paréntesis

- **Convenio 1:** Las flechas en los tipos se asocian por la derecha. Por ejemplo,

Int -> Int -> Int -> Int

representa a

```
Int -> (Int -> (Int -> Int))
```

- Convenio 2: Las aplicaciones de funciones se asocian por la izquierda. Por ejemplo,

```
mult x y z
```

representa a

$$((\text{mult } x) \ y) \ z$$

Nota: Todas las funciones con múltiples argumentos se definen en forma currificada, salvo que explícitamente se diga que los argumentos tienen que ser tuplas.

Ejemplo

```
between menor mayor nro = menor <= nro && nro <= mayor
```

```
> between 5 10 7
```

```
True
```

```
(between 18 65)
```

```
debeVotar persona = (between 18 65 . edad) persona
```

Ejemplo

```
map :: (a -> b) -> [a] -> [b]
```

```
> (map . length) ["Hola", "mundo"]
```

Ejemplo

```
map :: (a -> b) -> [a] -> [b]
```

```
> (map . length) ["Hola", "mundo"]
```

```
> (map length . filter (\e -> length e < 6))  
  ["hola", "palabraconmuchoscaracteres", "mundo"]
```

Parcialización

Parcialización

Idea

Parcializar es aplicar solo algunos argumentos de una función curried. El resultado es otra función, más especializada.

Distinción útil

- **Currificación:** propiedad de la forma de la función.
- **Parcialización:** uso concreto de esa forma.

Aporte práctico

Permite construir APIs declarativas: configurar una vez y reutilizar muchas veces.

```
inc :: Int -> Int
```

```
inc = suma 1
```

```
duplicar :: Int -> Int
```

```
duplicar = (* 2)
```

```
soloPares :: [Int] -> [Int]
```

```
soloPares = filter even
```

Orden superior

Funciones de orden superior

Definición

Una función es de **orden superior** si:

- recibe funciones como argumentos, o
- devuelve funciones como resultado.

Relación con lambda

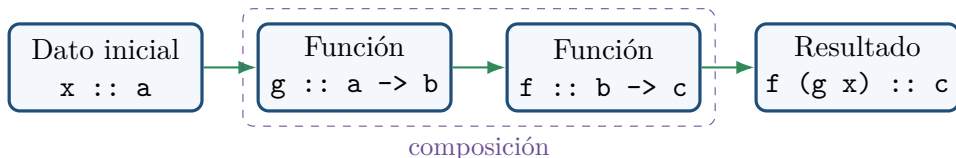
El cálculo lambda trata a las funciones como ciudadanos de primera clase.

Haskell toma esta idea y la vuelve cotidiana mediante tipos y combinadores reutilizables.

```
resultado :: [Int]
resultado = map (+1) (filter even [1..10])
-- [3,5,7,9,11]
```

```
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] ->
  ↳ [a]
foldr :: (a -> b -> b) -> b ->
  ↳ [a] -> b
```

Patrón de composición que aparece una y otra vez



Lo que habilitan currificación y orden superior

- pipelines legibles
- abstracciones pequeñas y combinables
- menos lógica repetida

Proyección a otros paradigmas

Estas ideas aparecen también en Scala, F#, Kotlin, JavaScript, Rust o Python cuando se usan callbacks, combinadores, streams o funciones anónimas.

Evaluación Lazy

Evaluación Impaciente y Perezosa

Función: `doble x = x + x`

- Ejemplo de evaluación anidada impaciente: `doble (doble 3)`

```
doble (doble 3) =
  = doble (3 + 3)    [def. de doble]
  = doble 6          [def. de +]
  = 6 + 6            [def. de doble]
  = 12               [def. de +]
```

- Ejemplo de evaluación anidada perezosa:

```
doble (doble 3) =
  = (doble 3) + (doble 3)    [def. de doble]
  = (3 + 3) + (doble 3)     [def. de doble]
  = 6 + (doble 3)           [def. de +]
  = 6 + (3 + 3)             [def. de doble]
  = 6 + 6                   [def. de +]
  = 12                      [def. de +]
```

Evaluación sobre listas

- Especificación: `sum xs` es la suma de los elementos de `xs`.
- Ejemplo: `sum [2,3,7] = 12`
- Definición:

```
sum []      = 0
sum (x:xs) = x + sum xs
```

- Evaluación:

```
sum [2,3,7] =
    = 2 + sum [3,7]           [def. de sum]
    = 2 + (3 + sum [7])       [def. de sum]
    = 2 + (3 + (7 + sum []))  [def. de sum]
    = 2 + (3 + (7 + 0))       [def. de sum]
    = 12                       [def. de +]
```

Evaluación eager vs lazy

Eager (estricta)

Evalúa argumentos **antes** de entrar a la función.

Ventaja: el costo es más predecible.

Riesgo: puede hacer trabajo innecesario.

Lazy / no estricta

Evalúa **solo cuando hace falta**.

Ventaja: permite modularidad, estructuras infinitas y cortocircuito natural.

Riesgo: puede ocultar costos o acumular thunks.

En Haskell

Haskell es **no estricto por defecto**.

En la práctica, implementa evaluación lazy con compartición: una expresión se calcula a demanda y luego se reutiliza.

Ejemplos de evaluación en Haskell

```
const1 :: a -> Int
const1 _ = 1
```

```
ok1 :: Int
ok1 = const1 undefined
-- resultado: 1
```

```
primeros :: [Int]
primeros = take 5 [1..]
-- [1,2,3,4,5]
```

```
shortCircuit :: Bool
shortCircuit = foldr (&&) True (False : undefined)
-- resultado: False
```

Qué muestran

- Si un argumento no se necesita, no se fuerza.
- Se pueden consumir prefijos finitos de estructuras infinitas.

Comparación mental con eager

En un lenguaje estricto, `const1 undefined` divergiría antes de entrar al cuerpo.

Qué aportan estas ideas al paradigma funcional y a otros estilos

En paradigma funcional

- **Composición:** resolver problemas uniendo piezas pequeñas.
- **Reutilización:** funciones generales + parcialización.
- **Seguridad:** tipos como Maybe hacen explícitos los casos límite.
- **Abstracción:** patrones como map, fold o bind evitan repetir estructura de control.

En paradigmas modernos

- **APIs funcionales:** streams, pipelines, callbacks, promesas.
- **Efectos controlados:** opciones, resultados, asincronía, logging.
- **Diseño declarativo:** describir el *qué* más que el *cómo*.

Ejemplificación

Funciones anónimas: JavaScript y Python

JavaScript

```
const cuadrados = [1, 2, 3].map(x => x * x);  
const suma = (x, y) => x + y;
```

```
console.log(cuadrados); // [1, 4, 9]  
console.log(suma(2, 3)); // 5
```

Python

```
cuadrados = list(map(lambda x: x * x, [1, 2, 3]))  
suma = lambda x, y: x + y
```

```
print(cuadrados) # [1, 4, 9]  
print(suma(2, 3)) # 5
```

Observación

Funciones anónimas: JavaScript y Python

Observación

En Python las `lambda` están limitadas a una sola expresión; en JavaScript las funciones flecha son más flexibles y comunes.

Funciones anónimas: Java, Go y Rust

Java

```
import java.util.function.Function;

Function<Integer,Integer> inc = x -> x + 1;
System.out.println(inc.apply(4));
```

Go

```
inc := func(x int) int {
    return x + 1
}
fmt.Println(inc(4))
```

Funciones anónimas: Java, Go y Rust

Rust

```
let inc = |x: i32| x + 1;  
println!("{}", inc(4));
```

Matiz

Java usa lambdas apoyadas en interfaces funcionales. Go y Rust tienen cierres (closures) más directos, aunque con reglas distintas sobre captura de variables.

Funciones de orden superior: la idea común

Definición

Una función es de **orden superior** si recibe otra función o devuelve una función.

Patrón típico

```
transformar(lista, f)
```

La lógica general recorre la colección y f decide qué hacer con cada elemento.

```
lista      = [1, 2, 3]
f(x)       = x * 2
resultado  = [2, 4, 6]
```

- `map`: transforma.
- `filter`: selecciona.
- `fold/reduce`: acumula.

Orden superior en JavaScript y Python

JavaScript

```
const pares = [1, 2, 3, 4].filter(x => x % 2 === 0);  
const total = [1, 2, 3, 4].reduce((acc, x) => acc + x, 0);
```

Lectura

filter recibe un predicado y **reduce** una función acumuladora.

Orden superior en JavaScript y Python

Python

```
pares = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4]))  
total = sum(map(lambda x: x * 2, [1, 2, 3]))
```

Lectura

En Python moderno a veces se prefiere comprensión de listas, pero `map` y `filter` siguen siendo ejemplos claros de orden superior.

Orden superior en Java, Go y Rust

Java

```
var xs = List.of(1, 2, 3, 4);  
var pares = xs.stream()  
    .filter(x -> x % 2 == 0)  
    .toList();
```

Go

```
func mapInt(xs []int, f func(int) int) []int {  
    ys := make([]int, len(xs))  
    for i, x := range xs { ys[i] = f(x) }  
    return ys  
}
```

Orden superior en Java, Go y Rust

Rust

```
let xs = vec![1, 2, 3, 4];  
let pares: Vec<_> = xs.into_iter()  
    .filter(|x| x % 2 == 0)  
    .collect();
```

Idea

Java y Rust traen utilidades de biblioteca muy ricas. En Go, la idea existe, pero muchas veces hay que construir el combinador explícitamente.

Aplicación parcial en JavaScript y Python

JavaScript

```
const suma = x => y => x + y;
const inc = suma(1);

console.log(inc(5)); // 6
```

Lectura

La aplicación parcial aparece de forma muy natural con cierres y funciones que devuelven funciones.

Python

```
from functools import partial

def suma(x, y):
    return x + y

inc = partial(suma, 1)
print(inc(5))    # 6
```

Lectura

Python ofrece `functools.partial`, una ayuda explícita para fijar argumentos.

Aplicación parcial en Java y Go

Java

```
import
↳ java.util.function.IntUnaryOperator

static IntUnaryOperator sumar(int x)
↳ {
    return y -> x + y;
}

var inc = sumar(1);
System.out.println(inc.applyAsInt(5));
```

Go

```
func sumar(x int) func(int) int {
    return func(y int) int {
        return x + y
    }
}

inc := sumar(1)
fmt.Println(inc(5))
```

Comentario

En ambos casos la aplicación parcial se consigue devolviendo una función que captura el valor ya fijado.

Aplicación parcial en Rust

```
fn sumar(x: i32) -> impl Fn(i32) -> i32
↳ {
    move |y| x + y
}

fn main() {
    let inc = sumar(1);
    println!("{}", inc(5));
}
```

Lectura

- `move` asegura que el cierre capture `x`.
- No hay un operador “mágico” de parcialización.
- El patrón idiomático es devolver un closure.

Comparación final

Más directos para estos patrones

- **JavaScript:** muy cómodo para callbacks y composición ligera.
- **Python:** expresivo, con apoyo extra de **partial**.
- **Rust:** muy potente con iteradores y closures.

Más verbosos

- **Java:** poderoso, pero con más sintaxis y tipos auxiliares.
- **Go:** soporta la idea, aunque suele requerir definir combinadores a mano.

Mensaje para llevar

Las ideas funcionales no son exclusivas de Haskell: aparecen en muchos lenguajes, pero cambian su **ergonomía**, su **idiomaticidad** y el grado en que el lenguaje las pone en primer plano.

Monadas

46/49

Maybe como mónada

```

procesoM :: Double -> Double -> Maybe
           ↳ Double
procesoM a b =
    safeDiv a b >=> safeRoot
  
```

```

procesoDo :: Double -> Double -> Double
           ↳ -> Maybe Double
procesoDo a b c = do
    q <- safeDiv a b
    r <- safeRoot q
    safeDiv r c
  
```

Qué resuelve

- Evita anidar case.
- Expresa el flujo principal y delega el manejo del contexto.
- Hace más legible la secuenciación.

Lectura de do

do es sintaxis azucarada para una cadena de `>=>`.

Leyes de mónadas y cierre conceptual

Leyes

```
return x >>= f ≡ f x
m >>= return ≡ m
(m >>= f) >>= g ≡ m >>= (\x -> f x >>= g)
```

Por qué importan

Garantizan que la secuenciación sea coherente: identidad y asociatividad para componer cálculos con contexto.

Cadena conceptual del deck

- ① funciones como valores
- ② currificación
- ③ parcialización
- ④ orden superior
- ⑤ evaluación no estricta
- ⑥ contexto explícito con Maybe
- ⑦ secuenciación genérica con mónadas

Resumen final

Idea central

El paradigma funcional gana potencia cuando combinamos **funciones pequeñas**, **tipos expresivos** y **composición controlada**.

- Maybe hace explícita la ausencia.
- La currificación convierte funciones multiargumento en cadenas de funciones unarias.
- La parcialización especializa funciones sin duplicar código.
- El orden superior vuelve natural la abstracción sobre comportamiento.
- La evaluación lazy cambia cuándo se computa.
- Las mónadas ordenan cálculos con contexto, especialmente útil en Haskell.

Mensaje para llevar

Si entiendes **aplicación**, **composición** y **tipos**, entiendes gran parte del núcleo del paradigma funcional moderno.